Project Methodology

Tiny code (SQL) - Fine-Tuning Lightweight Language Models for BigQuery SQL Generation

Sanjay Kumar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SanjayKumar@sandiego.edu

Santa Das Karmakar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SDasKarmakar@sandiego.edu

Sethuraman Krishnasamy
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SKrishnasamy@sandiego.edu

Project Methodology (AI590)

4 Methodology

This project implemented supervised fine-tuning of a large language model to generate syntactically correct, BigQuery-compliant SQL queries from natural language prompts augmented with schema context. The core model was the instruction-tuned Gemma-3-4b-it decoder-only transformer (Gemma Team, 2025), adapted via Low-Rank Adaptation (LoRA) (Hu et al., 2021) using the Hugging Face PEFT and Transformers libraries (Wolf et al., 2020).

Notable architectural and system design choices included LoRA adapters with rank 32, scaling factor $\alpha = 64$, dropout 0.05, and targeted modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj). These selections enabled parameter-efficient updates on only ~1–2 % of the 4-billion-parameter model while retaining the full pre-trained weights, making training feasible on a single NVIDIA L4 GPU within a cost-effective GKE environment. Sequence length was capped at 64 tokens to balance context retention with memory constraints during inference and training. The end-to-end pipeline was containerized in a custom Docker image pushed to Google Artifact Registry and executed as a Kubernetes Job on a GKE zonal cluster (sdk-training-cluster) equipped with a dedicated GPU node pool (g2-standard-16, 1× NVIDIA L4). Google Cloud Storage Fuse CSI provided seamless mounting of the prepared dataset, while

workload identity and service accounts ensured secure access to Hugging Face and GCS resources.

Data preparation formed the first stage of the pipeline: 100 000 training samples were drawn from the "train" split of the gretelai/synthetic_text_to_sql dataset (Gretel AI, 2024) and 5 000 held-out samples from the "test" split. Each example underwent deterministic preprocessing (dialect transpilation to BigQuery via SQLGlot (Mao, 2024), schema pruning with negative sampling, value-grounding hints, and multi-style prompt augmentation) before formatting into Gemma's chat template. The resulting JSONL files were uploaded to GCS and mounted read-only inside the training container.

Training followed a standard causal language modeling procedure. The dataset was tokenized on-the-fly with padding to the maximum length of 64; no additional manual split was required beyond the original train/test division. Batches of size 4 were processed with gradient accumulation steps of 4, yielding an effective batch size of 16. Training ran for 3 epochs (capped at 20 max steps for the demonstration configuration), using a linear learning-rate schedule with 0.1 warmup ratio, the paged_adamw_32bit optimizer, and the default cross-entropy loss for next-token prediction. Logging occurred every 5 steps.

During model optimization the LoRA rank, learning rate, batch size, and gradient accumulation values were explored across modest grids before finalizing the configuration shown in the project's config.yaml ($r = 32$, $\alpha = 64$, $lr = 2 \times 10^{-4}$, batch size = 4, accumulation = 4, epochs = 3). The production-grade pipeline—GKE provisioning via shell scripts, Docker build/push, ConfigMap injection, and Kubernetes Job deployment—ensured reproducibility and scalability for a simulated live AI product serving enterprise SQL-generation requests.

References

- Gemma Team. (2025). *Gemma 3 technical report*. arXiv. https://doi.org/10.48550/arXiv.2503.19786
- Gretel AI. (2024). *synthetic_text_to_sql* [Dataset]. Hugging Face. https://huggingface.co/datasets/gretelai/synthetic_text_to_sql

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*. https://arxiv.org/abs/2106.09685
- Mao, T. (2024). *SQLGlot: Python SQL parser and transpiler* [Software]. GitHub. https://github.com/tobymao/sqlglot
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., ... & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-demos.6